Fonaments d’Informàtica

4. Estructura del computador i tipus d’arquitectures

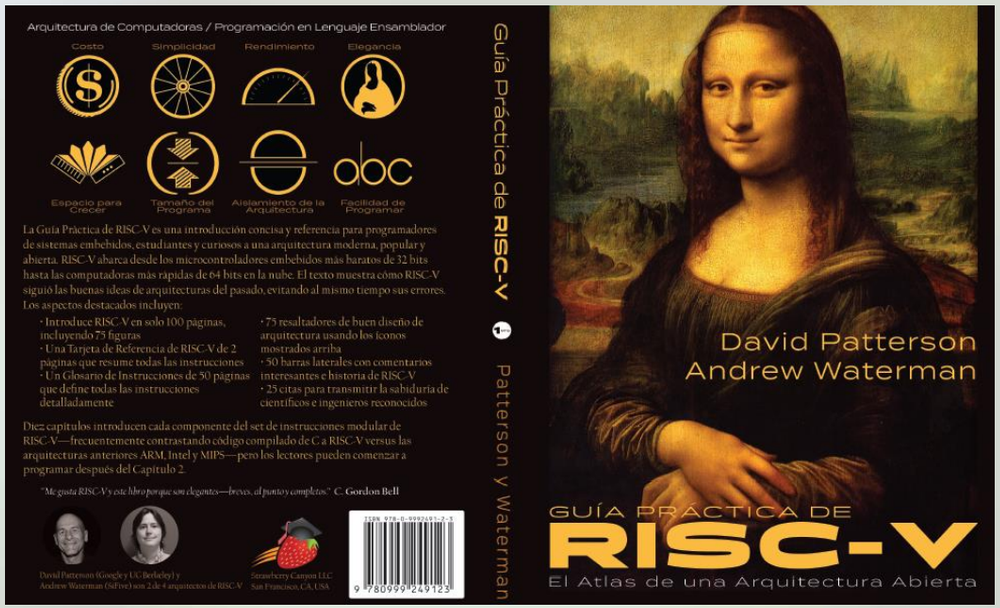
Enginyeria de Sistemes de Telecomunicació
Enginyeria Electrònica de Telecomunicació

Dpt. de Microelectrònica i Sistemes Electrònics

Curs 2026/27

UAB Universitat Autònoma de Barcelona

Bibliografia



Bàsica

D. Patterson, A. Waterman. *Guía práctica de RISC-V: el atlas de una arquitectura abierta*. 1a ed., 2018.

Cap. 1 → aquest tema · cap. 2 i 3 → Tema 5

Complementària

D. A. Patterson, J. L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. 2a ed., Morgan Kaufmann.

Cap. 1 (rendiment) i cap. 2 (ISA)

S. L. Harris, D. M. Harris. *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann.

Cap. 6 (arquitectura)

Índex

1 **Model de Von Neumann**

Programa emmagatzemat, coll d'ampolla, Harvard

2 **La CPU i el cicle d'instrucció**

UC, UP, PC, IR, fetch-decode-execute

3 **Rendiment**

T = N · CPI · Tc, CPI mitjà, MIPS

4 **La memòria vista per la CPU**

Adreces, paraules, alineació, endianness

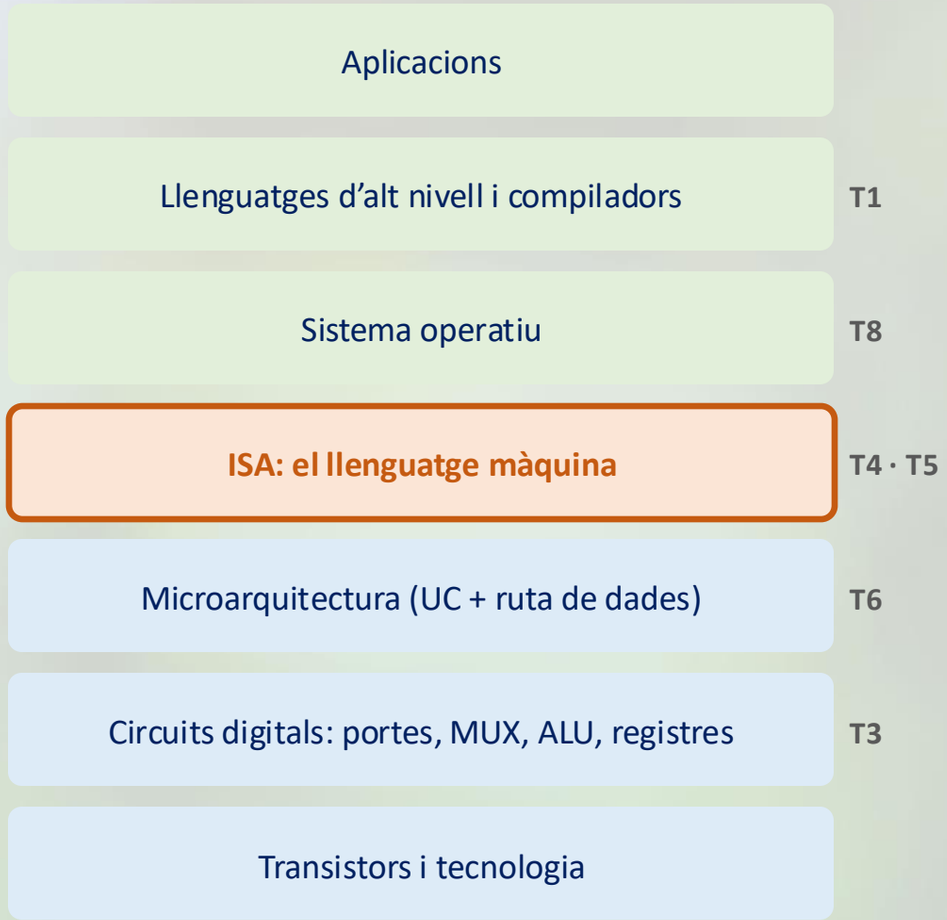
5 **Busos i interconnexió**

Adreces, dades i control; amplada de banda

6 **ISA: CISC i RISC**

Classificació, modes d'adreçament, CISC vs RISC

Introducció: on som?



- **Arquitectura del computador**: el disseny conceptual i el comportament funcional d'un computador, tal com el veu qui el programa.
- El llenguatge que entén la màquina és la seva **ISA** (*Instruction Set Architecture*): és la **frontera entre maquinari i programari**.
- En aquest tema: com s'organitza un computador (CPU, memòria, busos) i quins tipus d'ISA hi ha.
- Tema 5: programarem una ISA concreta (RISC-V).
  Tema 6: com està feta per dins.

1. Model de Von Neumann: el programa emmagatzemat

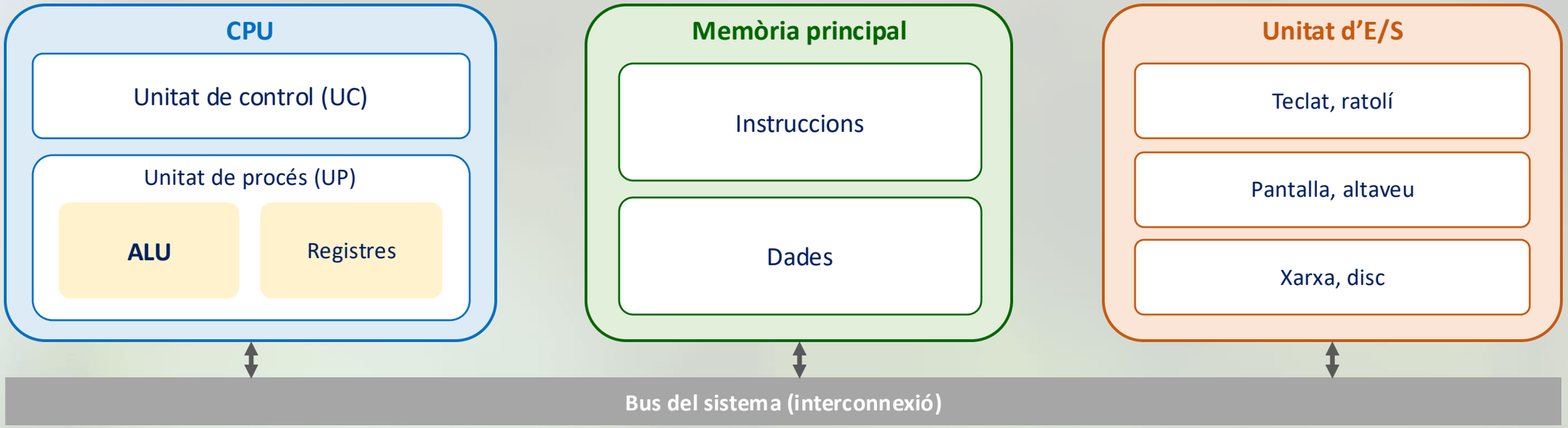
- 1945: *First Draft of a Report on the EDVAC* (J. von Neumann, amb Eckert i Mauchly) → vegeu el Tema 1.
- A l'ENIAC el programa es «cablejava» amb connexions i interruptors: canviar de programa costava dies.
- Idea clau: **el programa és una dada més**. Instruccions i dades es codifiquen en binari i es guarden a la **mateixa memòria**.
  - Canviar de programa = carregar uns altres bits a memòria.
  - Un programa pot llegir, modificar o crear programes (compiladors, SO, I els carregadors).
- La CPU llegeix les instruccions de memòria **una darrere l'altra** i les executa; només els salts trenquen aquest ordre.

Memòria principal

0x00	instr.: carrega A
0x04	instr.: suma B
0x08	instr.: desa a C
0x0C	instr.: salta a 0x00
0x10	dada A = 25
0x14	dada B = 17
0x18	dada C = ?

*Tot són bits a la mateixa memòria: el que distingeix una instrucció d'una dada és **com es fa servir**.*

1. Model de Von Neumann (II): les unitats



CPU

Executa les instruccions. La **UC** dirigeix i seqüencia; la **UP** opera amb les dades.

Memòria

Guarda instruccions i dades en paraules, cadascuna amb la seva **adreça**.

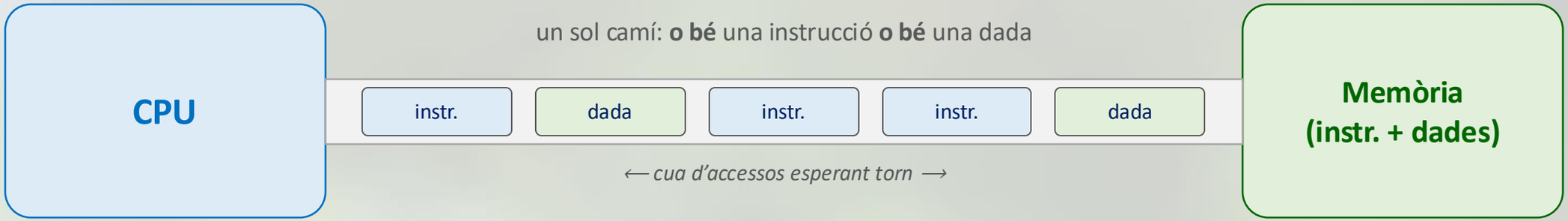
Entrada/sortida

Comunica amb l'exterior a través dels **perifèrics**.

Interconnexió

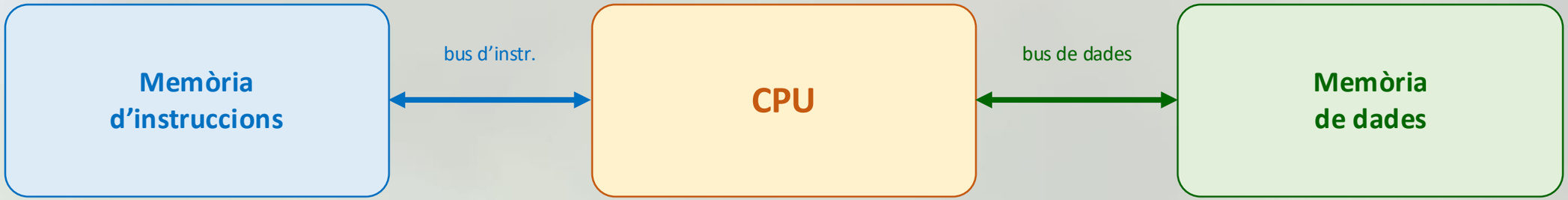
Busos que porten adreces, dades i senyals de control (apartat 5).

1. Model de Von Neumann (III): el coll d'ampolla



- Instruccions i dades viatgen pel **mateix camí** (bus) entre la CPU i la memòria.
- Cada instrucció necessita com a mínim un accés a memòria (portar-la), més els accessos de les seves dades.
- La CPU és molt més ràpida que la memòria → la CPU **espera**. És el *von Neumann bottleneck* (J. Backus, 1977).
- Solucions: **dos camins** separats (Harvard), **memòria cau** (Tema 7a) i busos més amples i ràpids.

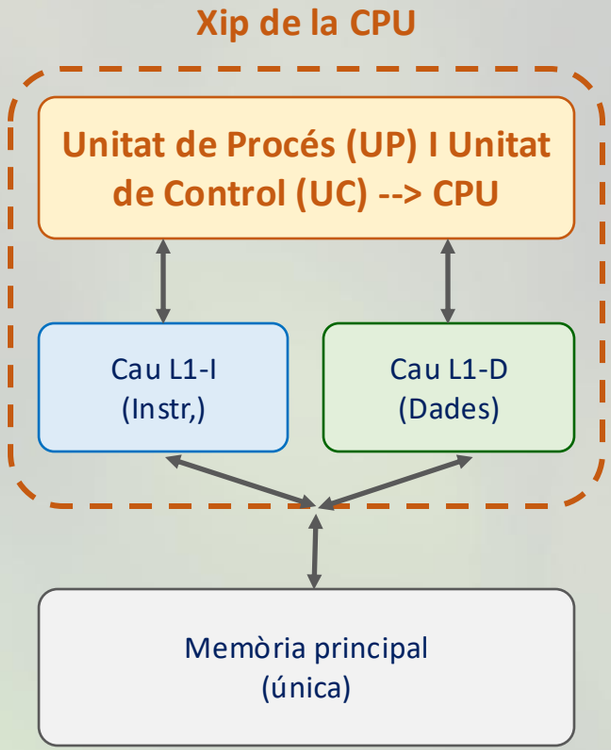
1. Arquitectura Harvard



- Variant de Von Neumann: programa i dades en **memòries separades**, amb **busos separats**.
  - Els espais d'adreces i l'amplada de paraula de cada memòria poden ser diferents.
  - El principi de funcionament (cicle d'instrucció) és el mateix.
- Pot **llegir una instrucció i una dada alhora** → més velocitat, menys congestió.
- Inconvenient: cal dimensionar bé cada memòria (la que sobra en una no es pot fer servir a l'altra).
- Molt habitual en **microcontroladors** (AVR d'Arduino, PIC) i en **DSP** (processadors de senyal: filtres, mòdems, àudio).

1. Von Neumann vs Harvard: la solució actual

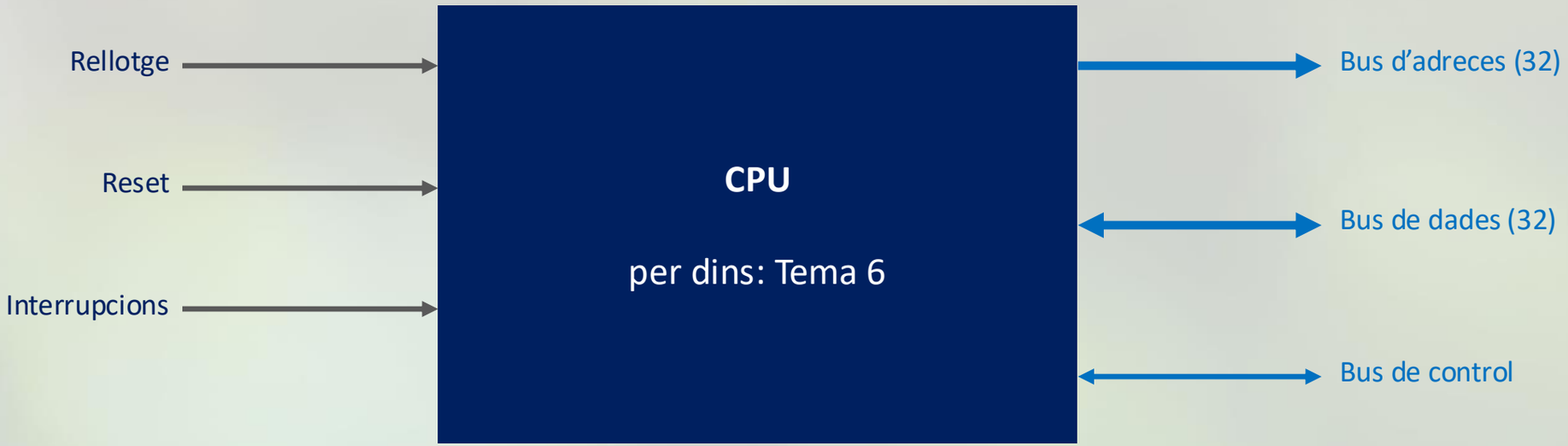
	Von Neumann	Harvard	Harvard modificada
Memòries	1 compartida	2 separades	1 principal + 2 caus L1 (I i D)
Camins (busos)	1	2	2 fins a les caus, 1 a memòria
Instr. i dada alhora	No	Sí	Sí (si són a la cau)
Programa com a dada	Sí	Difícil	Sí
Exemples	Primers computadors, sistemes senzills	Microcontroladors, DSP	PC, mòbils, servidors (x86, ARM, RISC-V)



Harvard a prop de la CPU, Von Neumann a la memòria.

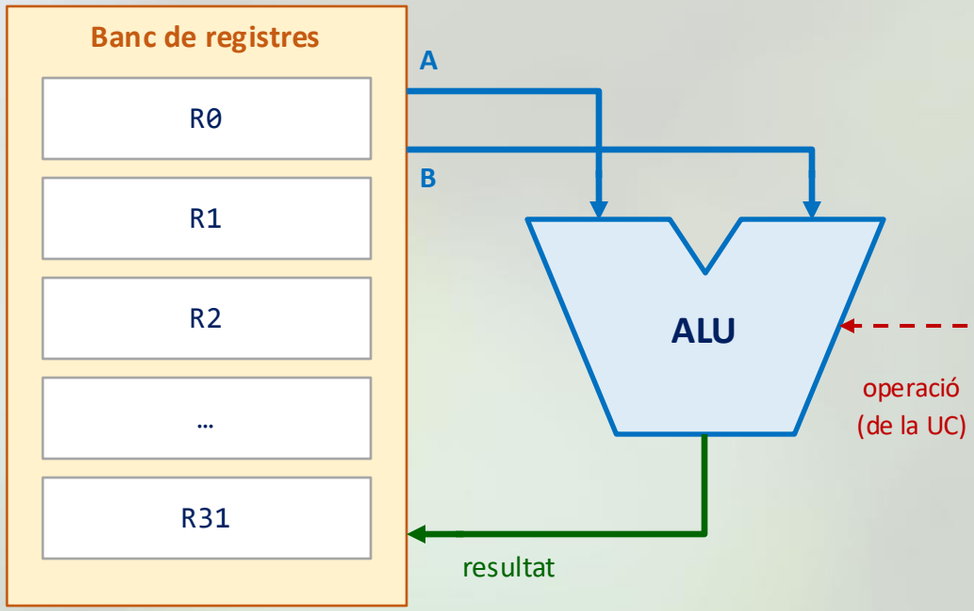
Els processadors actuals són **Von Neumann** vistos pel programador (una sola memòria), però per dins tenen camins **Harvard** gràcies a les memòries cau separades → Tema 7a.

2. La CPU (I): vista com una capsa negra



De moment no ens interessa com està feta per dins (team 6), sinó **què fa**:
llegeix instruccions de memòria, les executa i torna a començar. Es comunica amb la resta del sistema a través dels **busos**.

2. La CPU (II): la unitat de procés



- **ALU** (Tema 3): suma, resta, AND, OR, XOR, desplaçaments, comparacions. Quina operació fa ho decideix la UC.
- **Banc de registres**: petita memòria **molt ràpida** dins la CPU (a RISC-V, 32 registres de 32 bits).
  - En un cicle se'n poden llegir dos (operands A i B) i escriure'n un (resultat).
  - Accedir a un registre (1 cicle de rellotge) és unes 100 vegades més ràpid que accedir a la memòria principal.
- Les dades es porten de memòria a registres, s'hi opera i, si cal, es tornen a desar.

2. La CPU (III): registres especials

PC — **Program Counter (comptador de programa)**

- Conté l'**adreça** de la propera instrucció.
- Normalment PC ← PC + 4 (cada instrucció ocupa 4 bytes).
- Els **salts** hi escriuen una altra adreça.

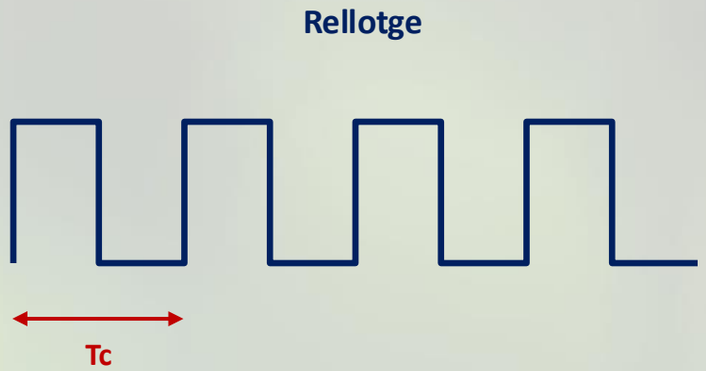
IR — **Instruction Register (registre d'instrucció)**

- Conté la **instrucció** que s'està executant (els seus bits).
- La UC la llegeix per saber què ha de fer.
- El programador no hi té accés.

I el registre d'estat (flags)?
Moltes ISA (x86, ARM) tenen un registre d'estat amb indicadors que actualitza l'ALU: Z (zero), N (negatiu), C (carry), V (overflow). Els salts condicionals els consulten.
RISC-V no té registre d'estat: els salts condicionals comparen directament dos registres (p. ex. «salta si x1 = x2»).

2. La CPU (IV): unitat de control i rellotge

- **Unitat de control (UC)**: llegeix el codi d'operació (*opcode*) de l'IR i genera els **senyals de control**:
  - quina operació fa l'ALU i quins registres es llegeixen o s'escriuen;
  - si es llegeix o s'escriu a memòria;
  - quin és el proper valor del PC.
- També **seqüencia** les fases de cada instrucció. Per ara és una capsa negra.
- **Rellotge**: senyal periòdic que marca el ritme de tota la CPU. Els canvis es fan al flanc.
  - Període **Tc** (temps de cicle) i freqüència **f = 1/Tc**.
  - Exemple: f = 2 GHz → Tc = 1 / ($2 \cdot 10^9$) = 0,5 ns.
- Una instrucció pot necessitar **un o diversos cicles** de rellotge.

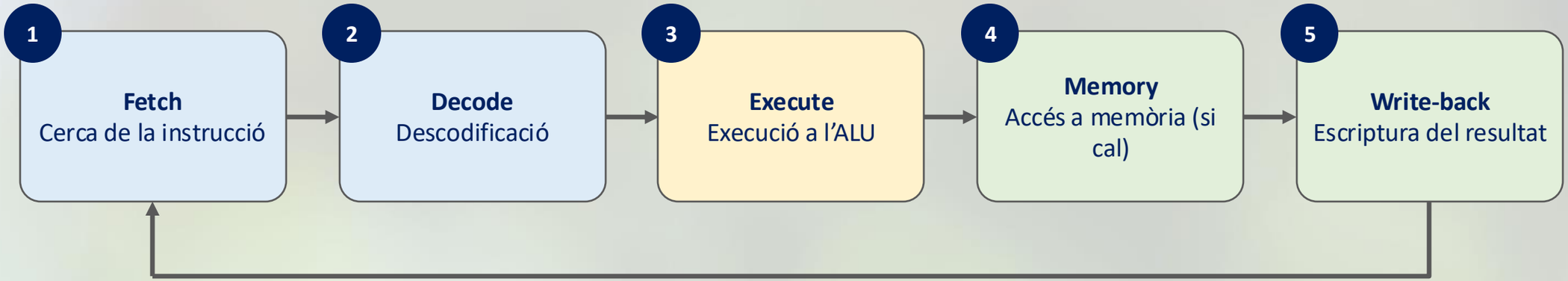


f = 1 / Tc

1 GHz ↔ 1 ns

100 MHz ↔ 10 ns

2. El cicle d'instrucció



el PC ja apunta a la instrucció següent → es torna a començar

La CPU no fa res més: des que s'engega fins que s'apaga, repeteix aquest cicle per a cada instrucció.

Les fases 1 i 2 són iguals per a totes les instruccions; les 3, 4 i 5 depenen de la instrucció.

Model de 3 fases
Sovint s'agrupen les fases 3, 4 i 5 en una sola, «execute»:

fetch → decode → execute

Les 5 fases separades són la base de la **segmentació** (Tema 6).

2. El cicle d'instrucció (II): les fases en detall

Fase	Què passa	RTL
1. Fetch	La CPU envia el PC a memòria, llegeix la instrucció i avança el PC	IR ← M[PC] PC ← PC + 4
2. Decode	La UC interpreta l'opcode i llegeix els registres font	A ← R[rs1] B ← R[rs2]
3. Execute	L'ALU opera, calcula una adreça o compara dos valors	res ← A op B
4. Memory	Només si la instrucció llegeix o escriu memòria	dada ← M[res] o bé M[res] ← B
5. Write-back	Es desa el resultat en un registre	R[rd] ← res (o dada)

Notació RTL (*Register Transfer Level*): X ← Y vol dir «el valor de Y es copia a X».
M[a]: la posició de memòria d'adreça a. R[i]: el registre i. PC + 4: cada instrucció ocupa 4 bytes.

2. El cicle d'instrucció (III): exemple pas a pas

Memòria (C = A + B)

0x00	`R1 ← M[0x40]`
0x04	`R2 ← M[0x44]`
0x08	`R3 ← R1 + R2`
0x0C	`M[0x48] ← R3`
…	
0x40	`A = 7`
0x44	`B = 5`
0x48	`C = ?`

PC	IR (fetch)	Execute	Canvis
0x00	R1 ← M[0x40]	llegeix memòria 0x40	R1 = 7, PC = 0x04
0x04	R2 ← M[0x44]	llegeix memòria 0x44	R2 = 5, PC = 0x08
0x08	R3 ← R1 + R2	ALU: 7 + 5	R3 = 12, PC = 0x0C
0x0C	M[0x48] ← R3	escriu memòria 0x48	C = 12, PC = 0x10

Quants accessos a memòria?
4 per portar les instruccions (fetch) + 3 per a les dades = **7**.
Més de la meitat del trànsit pel bus (4 de 7) són instruccions: aquest és el coll d'ampolla de Von Neumann.

2. El cicle d'instrucció (IV): un salt i un bucle

Programa (suma 3 + 2 + 1)

```
0x00   R1 ← 0
0x04   R2 ← 3
0x08   R1 ← R1 + R2
0x0C   R2 ← R2 – 1
0x10   si R2 ≠ 0: PC ← 0x08
0x14   M[0x40] ← R1
```

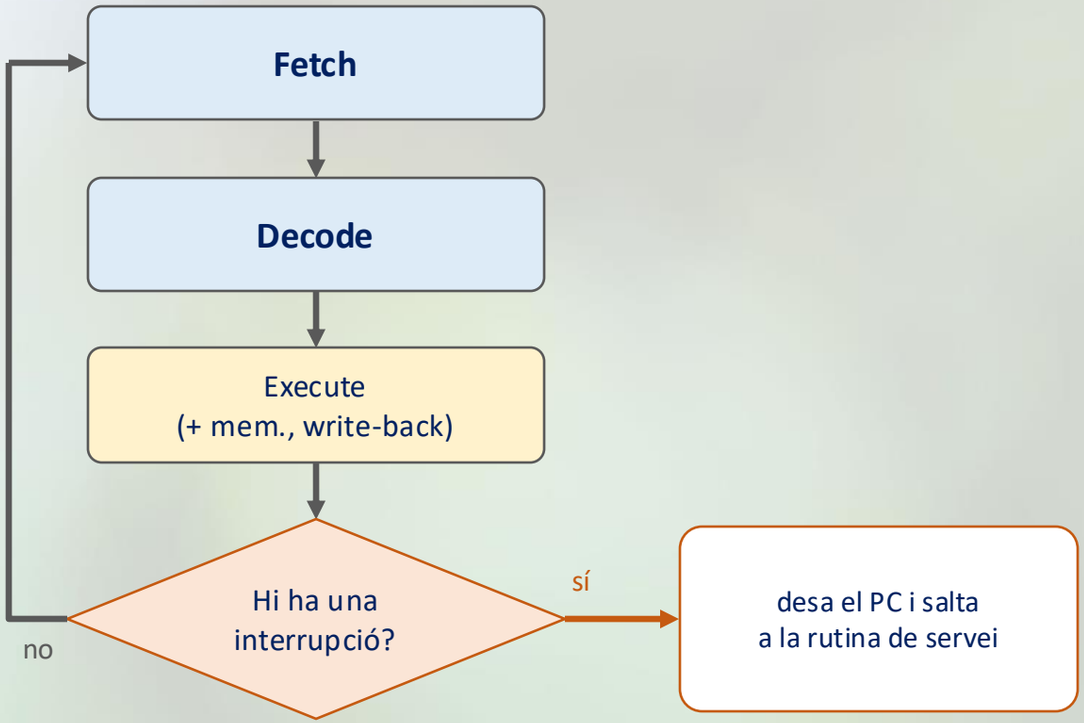
Salt condicional
si la condició es compleix: PC ← adreça de destí
si no: PC ← PC + 4 (com sempre)

Un salt **només canvia el PC**: la propera fase fetch ja llegeix d'una altra adreça.

PC	Instrucció	R1	R2	PC següent
0x00	R1 ← 0	0	–	0x04
0x04	R2 ← 3	0	3	0x08
0x08	R1 ← R1 + R2	3	3	0x0C
0x0C	R2 ← R2 – 1	3	2	0x10
0x10	si R2 ≠ 0 …	3	2	**0x08** (salta)
…	…	…	…	…

Exercici: completa la traça.
a) Quines adreces pren el PC fins al final?
b) Quantes instruccions s'executen?
c) Quants accessos a memòria es fan?

2. El cicle d'instrucció (V): i les interrupcions?



- Un perifèric (teclat, xarxa, temporitzador…) pot **avisar** la CPU activant una línia d'**interrupció**.
- La CPU ho comprova **al final de cada instrucció**, mai a mitges.
- Si n'hi ha una: desa el PC, executa la **rutina de servei** i després continua on era.
- Així la CPU no ha d'estar preguntant contínuament als perifèrics.

3. Rendiment (I): l'equació fonamental

$$T = N \times CPI \times Tc = N \times CPI / f$$

N Nombre d'instruccions **executades** (les dels bucles compten tantes vegades com s'executen).

CPI Cicles per instrucció (mitjana).

Tc, f Període (s) i freqüència (Hz) del rellotge.

Factor	Hi influeixen...
N	l'algorisme, el llenguatge, el compilador i la **ISA**
CPI	la **ISA**, la microarquitectura i la memòria
f	la tecnologia i la microarquitectura

Rendiment = 1 / T

A és n vegades més ràpid que B si T(B) / T(A) = n

3. Rendiment (II): exemple resolt

El mateix programa executa **$N = 2 \cdot 10^9$** instruccions en dos processadors:

Processador A

f = 2 GHz, CPI = 2

$T = 2 \cdot 10^9 \times 2 / 2 \cdot 10^9$

T = 2 s

Processador B

f = 3 GHz, CPI = 4

$T = 2 \cdot 10^9 \times 4 / 3 \cdot 10^9$

T ≈ 2,67 s

A és 2,67 / 2 ≈ **1,33 vegades més ràpid** que B, tot i tenir menys freqüència.
La freqüència sola **no** diu quin processador és més ràpid: cal mirar els tres factors.

3. Rendiment (III): CPI mitjà

No totes les instruccions tarden el mateix. Si f_i és la fracció d'instruccions de la classe i:

$$\mathbf{CPI = \Sigma\, f_i \times CPI_i}$$

Classe	Freqüència f_i	CPI_i	$f_i \times CPI_i$
Aritmètiques i lògiques	50 %	1	0,5
Accés a memòria	30 %	2	0,6
Salts	20 %	3	0,6
Total	100 %		**CPI = 1,7**

Exercici
Amb una memòria cau millor, els accessos a memòria passen a CPI = 1.

a) Quin és el nou CPI?
b) Quantes vegades més ràpid és el programa (mateixos N i f)?

3. Rendiment (IV): per què els MIPS enganyen

$$\text{MIPS} = N / (T \times 10^6) = f / (CPI \times 10^6)$$

Milions d'instruccions per segon: és fàcil de calcular, però **no té en compte N**.

El mateix programa, compilat per a dues màquines a f = 2 GHz:

	N	CPI	T = N·CPI/f	MIPS
Màquina X	$1 \cdot 10^9$	2	**1 s**	1000
Màquina Y	$3 \cdot 10^9$	1	**1,5 s**	**2000**

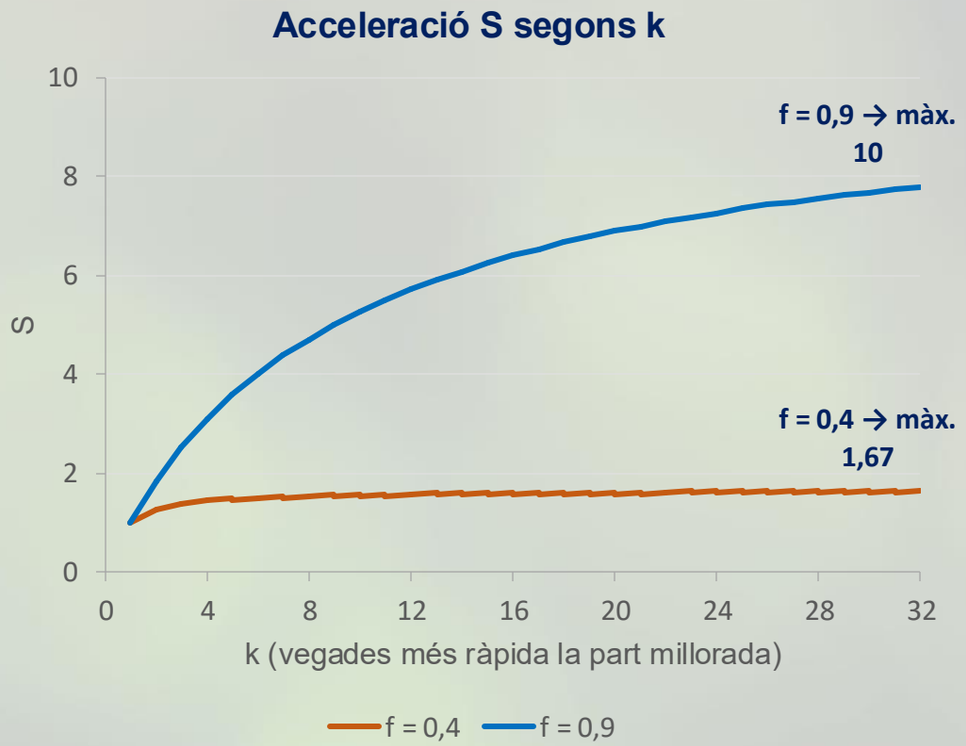
Y té **el doble de MIPS** però és **més lenta**: necessita més instruccions per fer la mateixa feina.
L'única mesura fiable és el **temps d'execució** de programes reals (*benchmarks*, p. ex. SPEC CPU).
Aquest és justament el debat CISC vs RISC.

3. Rendiment (V): la llei d'Amdahl

$$S = 1 / ((1 - f) + f / k)$$

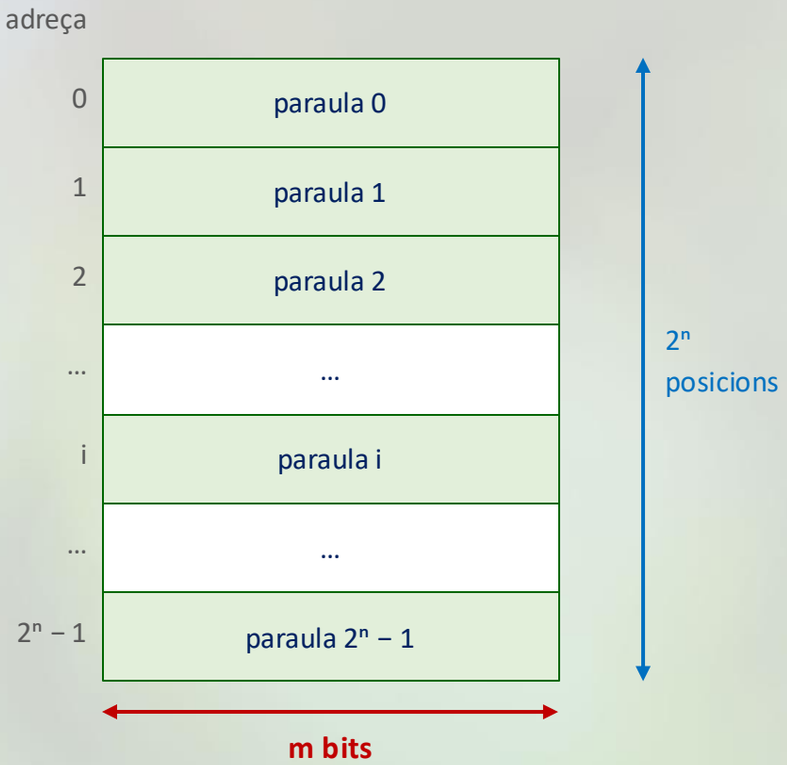
- **S** Acceleració (*speedup*): T abans / T després.
- **f** Fracció del temps d'execució que es pot millorar.
- **k** Quantes vegades més ràpida fem aquesta part.

Exemple: un programa passa el **40 %** del temps multiplicant.
Les fem **10 vegades** més ràpides (f = 0,4, k = 10):
S = 1 / (0,6 + 0,4/10) = 1 / 0,64 ≈ **1,56**
I si fossin infinitament ràpides (k → ∞)? S ≤ 1 / 0,6 ≈ **1,67**



Millora el cas comú: la part que no es millora (1 – f) acaba posant el límit. Per això 8 nuclis no fan un programa 8 vegades més ràpid, i per això la memòria pesa tant en el rendiment.

4. Memòria (I): un vector de posicions



- Per a la CPU, la memòria és un **vector**: cada posició té una **adreça** i guarda una paraula de m bits.
- Amb **n bits d'adreça** es poden adreçar **2^n posicions**. Capacitat = $2^n \times m$ bits.
- La **paraula** de la CPU sol ser de 32 bits (4 bytes) o 64 bits (8 bytes).
  - Adreces de 32 bits → 2^{32} bytes = **4 GiB**.
  - Adreces de 64 bits → 2^{64} bytes = 16 EiB ≈ $1{,}8 \cdot 10^{19}$ bytes (teòric).
- Com està feta per dins (SRAM, DRAM…)

4. Memòria (II): adreçament per byte

Memòria de 16 bytes, paraules de 32 bits

0x00	byte 0	P0	0x08	byte 8	P2
0x01	byte 1		0x09	byte 9	
0x02	byte 2		0x0A	byte 10	
0x03	byte 3		0x0B	byte 11	
0x04	byte 4	P1	0x0C	byte 12	P3
0x05	byte 5		0x0D	byte 13	
0x06	byte 6		0x0E	byte 14	
0x07	byte 7		0x0F	byte 15	

- Les adreces són **de byte**: cada byte té la seva adreça.
- Una paraula de 32 bits ocupa **4 adreces consecutives**.
- L'adreça d'una paraula és la del seu **primer byte** (la més baixa) → la paraula i és a l'adreça 4·i.
- **Exercici**: en aquesta memòria de 16 bytes, quantes paraules hi ha i a quines adreces comencen si la paraula és de 8, de 32 o de 64 bits?

4. Memòria (III): alineació

Una dada de k bytes està **alineada** si la seva adreça és **múltiple de k**.

Sense alinear (17 bytes)

	+0	+1	+2	+3
1000	a	b	c	h0
1004	h0	h1	h1	h2
1008	h2	w0	w0	w0
1012	w0	w1	w1	w1
1016	w1			

Alineat (20 bytes)

	+0	+1	+2	+3
1000	a	b	c	–
1004	h0	h0	h1	h1
1008	h2	h2	–	–
1012	w0	w0	w0	w0
1016	w1	w1	w1	w1

a, b, c: 1 byte · h0, h1, h2: 2 bytes · w0, w1: 4 bytes, a partir de l'adreça 1000. «–»: bytes de farciment.

- La memòria es llegeix per **paraules senceres**: una paraula no alineada (w0 a 1009) ocupa **dues** paraules de memòria → **dos accessos**.
- En algunes ISA un accés no alineat és **lent**; en altres provoca una **excepció**.
- Per això el compilador **alinea** les dades i hi deixa bytes de farciment: es perd una mica d'espai a canvi de velocitat.

4. Memòria (IV): ordre dels bytes (endianness)

Com es desa la paraula **0x40300C01** a l'adreça **0x18**?

	Big-endian
0x18	40
0x19	30
0x1A	0C
0x1B	01

	Little-endian
0x18	01
0x19	0C
0x1A	30
0x1B	40

Exercici
Com queda 0x12345678 a l'adreça 0x100 en cada cas?

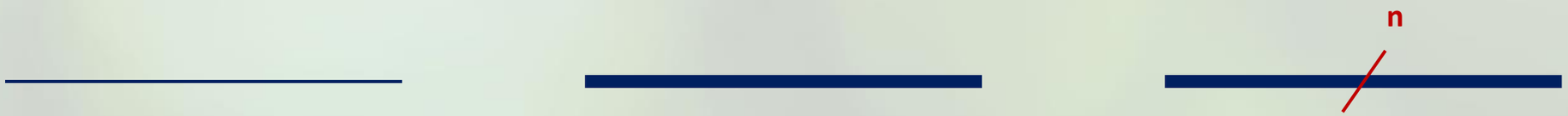
Quin byte llegeix la CPU a l'adreça 0x101?

- **Big-endian**: el byte **més** significatiu a l'adreça més baixa (com l'escrivim).
- **Little-endian**: el byte **menys** significatiu a l'adreça més baixa.
- RISC-V, x86 i ARM (normalment) són **little-endian**.
- Les xarxes TCP/IP envien els enters en **big-endian** (*network byte order*). Per enviar un enter des d'un PC cal convertir-lo (en C: `htonl()`, `ntohl()`).

5. Busos (I): què és un bus

- Un **bus** és un conjunt de **línies** (fils) compartides que transporten bits **en paral·lel** entre els components.
- L'**amplada** del bus és el nombre de línies: un bus de 32 bits porta una paraula sencera de cop.
- És **compartit**: només un component pot escriure-hi alhora (cal un **arbitratge**); tots els connectats el poden llegir.

Notació



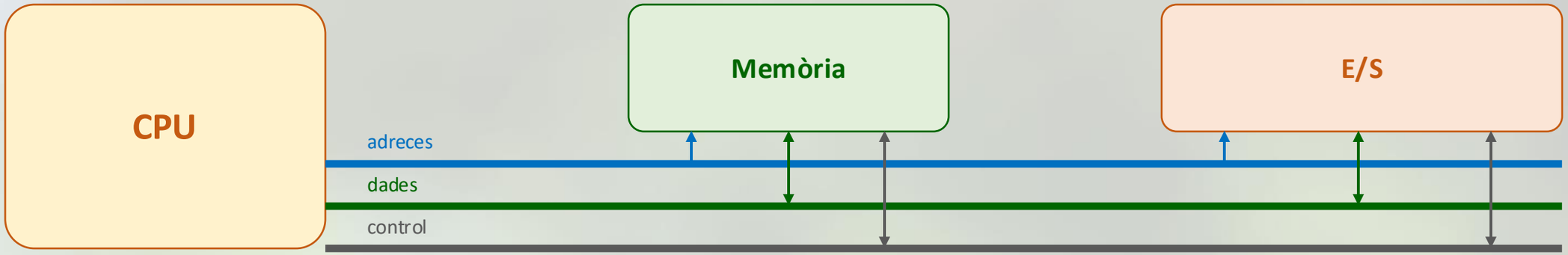
línia simple: 1 bit

línia gruixuda: bus

bus de n bits

Exemple: D = $\{d_{31}, \ldots, d_0\}$ viatja per un bus de dades de 32 línies.

5. Busos (II): adreces, dades i control

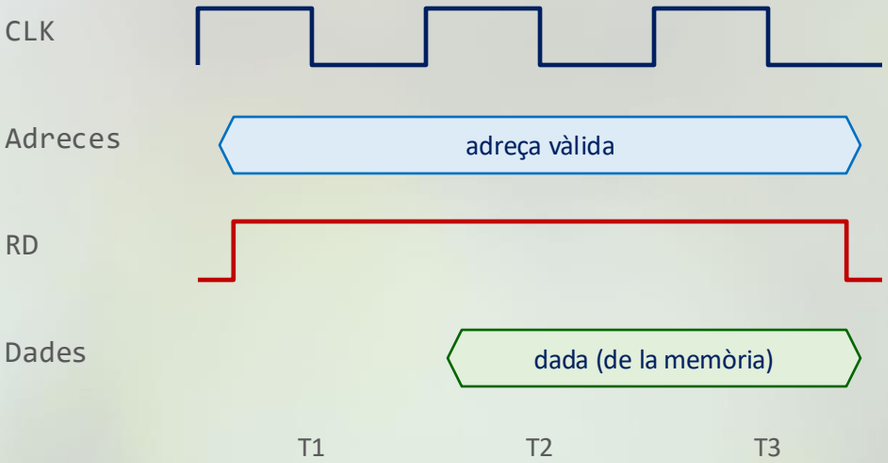


Bus	Sentit	Què porta	La seva amplada determina...
Adreces	CPU → memòria, E/S	L'adreça de la posició a llegir o escriure	la **memòria màxima**: n bits → 2^n adreces
Dades	bidireccional	La dada llegida o escrita	quants **bits per transferència**
Control	bidireccional	RD, WR, rellotge, interrupcions, petició de bus...	quines operacions es poden fer

Exemple: l'Intel 8086 tenia un bus d'adreces de 20 bits (1 MiB) i un de dades de 16 bits. **Exercici**: quina memòria màxima es pot adreçar amb 24 bits d'adreça?

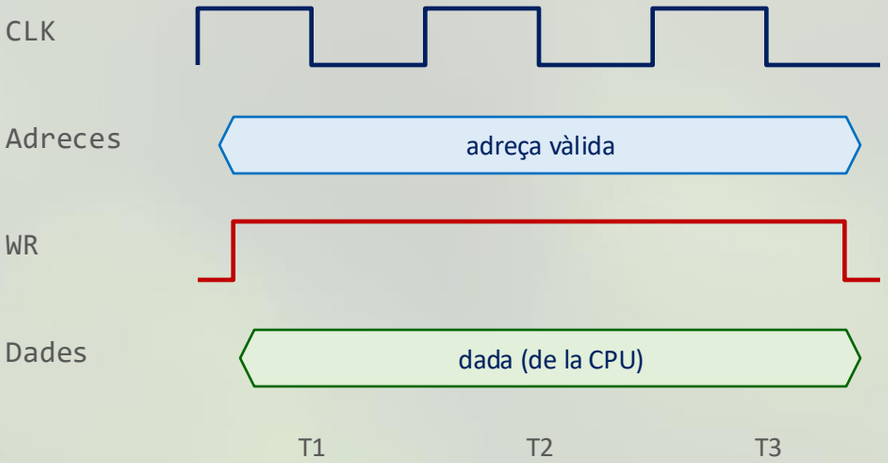
5. Busos (III): una lectura i una escriptura

Lectura (la CPU llegeix de memòria)



1. La CPU posa l'**adreça** al bus i activa **RD**.
2. La memòria busca la dada (tarda un temps).
3. La memòria posa la **dada** al bus; la CPU la captura i desactiva RD.

Escriptura (la CPU escriu a memòria)



1. La CPU posa l'**adreça** i la **dada** al bus i activa **WR**.
2. La memòria desa la dada a la posició indicada.
3. La CPU desactiva WR i allibera el bus.

Si la memòria és lenta s'afegeixen **cicles d'espera**.

5. Busos (IV): amplada de banda

BW = amplada (bytes) × f (Hz) × transferències per cicle

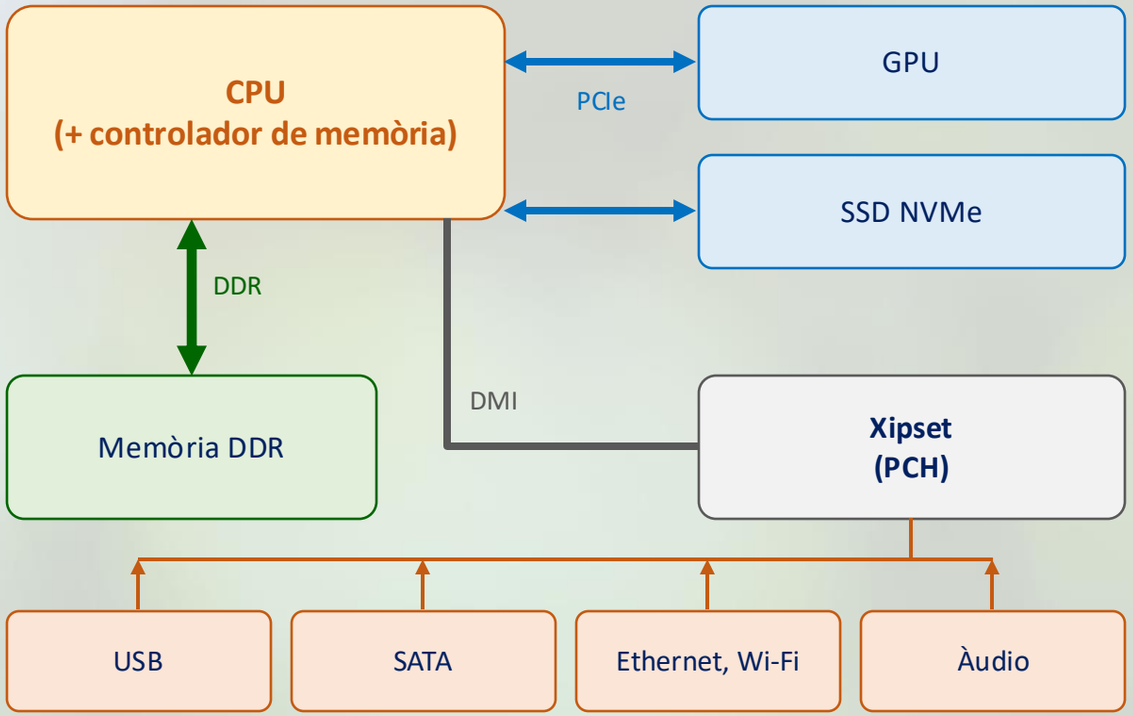
Bus / enllaç	Càlcul	BW
DDR4-3200 (1 canal)	8 B × 1600 MHz × 2	25,6 GB/s
PCIe 4.0, 1 carril	16 GT/s, sèrie	≈ 2 GB/s
PCIe 4.0 ×16	16 carrils	≈ 32 GB/s

Per què els enllaços moderns són sèrie (PCIe, USB, SATA)?
A freqüències altes, en un bus paral·lel els bits de cada línia arriben desfasats (*skew*) i hi ha diafonia. Un enllaç sèrie diferencial pot anar molt més de pressa.

Exercici
Un bus de 32 bits a 100 MHz fa una transferència per cicle.

a) Quina amplada de banda té?
b) Quant tarda a transferir un bloc de 4 KiB?

5. Busos (V): del bus únic a una jerarquia



Vegeu la placa base al Tema 1 (S15).

- Un sol bus per a tot seria un **coll d'ampolla**: dispositius lents frenarien els ràpids.
- Cada tipus de trànsit té el seu enllaç, a la velocitat que necessita.
- **E/S mapada a memòria**: els registres dels controladors de perifèrics tenen **adreces** com la memòria i la CPU hi accedeix amb les instruccions normals de lectura i escriptura (RISC-V ho fa així).
- Alternativa: E/S aïllada, amb instruccions especials (x86: `IN`, `OUT`).
- Tema 7b: com se sincronitzen CPU i perifèrics.

6. ISA (I): el contracte entre maquinari i programari

Programari: compiladors, sistema operatiu, programes

ISA

Maquinari: el processador i la memòria

Registres

quants n'hi ha, de quina mida i per a què serveixen

Tipus de dades

enters, coma flotant, mides (byte, paraula...)

Instruccions

transferència, aritmètiques i lògiques, control de flux

Modes d'adreçament

com s'indica on és cada operand

Format i codificació

com es converteix cada instrucció a bits

Model de memòria

adreçament per byte, alineació, endianness

És tot el que necessita saber qui programa en assemblador o qui escriu un compilador. No diu res de com està construït el processador.

6. ISA (II): arquitectura vs microarquitectura

Arquitectura (ISA) · QUÈ fa

- Visible per al programador.
- Instruccions, registres, memòria.
- Si dos processadors tenen la mateixa ISA, executen els mateixos programes.

Microarquitectura · COM ho fa

- Invisible per al programador.
- Segmentació, memòries cau, nombre d'ALU, freqüència...
- Determina el CPI i la f (i el consum i el preu).

ISA	Microarquitectures (implementacions) diferents
x86-64	Intel Core, Intel Atom, AMD Ryzen (Zen): el mateix executable funciona a tots
ARMv8 / ARMv9	Apple M, Qualcomm Snapdragon, Cortex-A de la Raspberry Pi
RISC-V	Des de microcontroladors (ESP32-C3) fins a processadors d'aplicació (SiFive)

Al Tema 6 dissenyarem a nivell de blocs una microarquitectura senzilla per a RV32I.

6. Tipus d'ISA (I): on són els operands?

La mateixa operació **C = A + B** (A, B i C són a memòria) en quatre tipus de màquina:

Pila	Acumulador	Registre-memòria	Load-store

`PUSH A`
`PUSH B`
`ADD`
`POP C`
	`LOAD A`
	`ADD B`
	`STORE C`
		`LOAD R1, A`
		`ADD R3, R1, B`
		`STORE R3, C`
			`LOAD R1, A`
			`LOAD R2, B`
			`ADD R3, R1, R2`
			`STORE R3, C`

Els operands són al cim d'una pila.	Un operand és sempre el registre acumulador.	Un operand pot ser directament a memòria.	Només LOAD/STORE accedeixen a memòria; l'ALU opera amb registres.
Ex.: JVM, calculadores RPN, x87	*Ex.: Primers computadors, micros de 8 bits*	*Ex.: x86, IBM 360, 68000*	*Ex.: RISC-V, ARM, MIPS*

Totes les ISA modernes de propòsit general són de **registres** (registre-memòria o load-store), perquè els registres són molt més ràpids que la memòria.

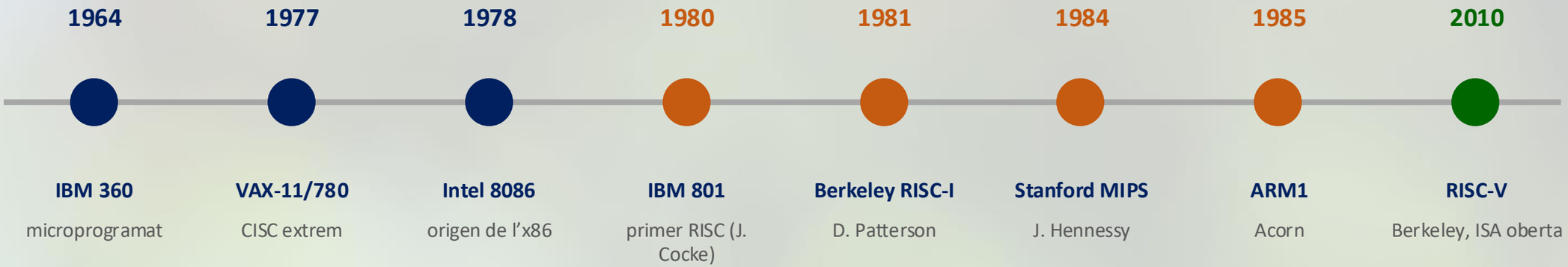
6. Tipus d'ISA (II): modes d'adreçament

El **mode d'adreçament** indica on és l'operand i com se'n calcula l'adreça.

Mode	Exemple (RTL)	On és l'operand	RISC-V
Immediat	R1 ← R2 + 5	a la mateixa instrucció (constant)	Sí (`addi`)
Registre	R1 ← R2 + R3	en un registre	Sí (`add`)
Directe (absolut)	R1 ← M[0x1000]	a memòria; l'adreça és a la instrucció	No
Indirecte per registre	R1 ← M[R2]	a memòria; el registre conté l'adreça (punter)	Sí (base + 0)
Base + desplaçament	R1 ← M[R2 + 8]	a memòria, a l'adreça registre + constant	Sí (`lw`, `sw`)
Relatiu al PC	PC ← PC + 16	instrucció a una distància del PC actual	Sí (salts)
Indirecte per memòria	R1 ← M[M[R2]]	a memòria hi ha l'adreça de l'operand	No (VAX sí)

RISC-V té **pocs modes**; les ISA CISC (x86, VAX) en tenen molts. Al Tema 5 veurem els de RISC-V amb la seva sintaxi.

6. CISC i RISC (I): d'on surten



CISC (*Complex Instruction Set Computer*)
- Memòria cara i lenta → **codi compacte**.
- Es programava molt en assemblador → instruccions «potents», a prop de l'alt nivell.
- La microprogramació feia barat afegir instruccions.

RISC (*Reduced Instruction Set Computer*)
- Els compiladors feien servir sobretot les instruccions **simples**.
- Millor fer poques instruccions simples **molt ràpides** (CPI ≈ 1).
- Memòries cau i compiladors millors ho fan possible.

Patterson i Hennessy: Premi Turing 2017.

6. CISC i RISC (II): comparativa

	CISC (x86)	RISC (RISC-V)
Instruccions	moltes (centenars)	poques (RV32I: ~40)
Longitud	variable (x86: 1 a 15 bytes)	fixa (32 bits)
Accés a memòria	moltes instruccions	només load i store
Modes d'adreçament	molts	pocs
Registres	pocs (x86: 8; x86-64: 16)	molts (32)
Cicles per instrucció	variable, sovint molts	≈ 1
Mida del codi	compacte (N baix)	més llarg (N alt)
Complexitat a…	el maquinari	el compilador

M[R + 8] ← M[R + 8] + 5

x86 (1 instrucció)

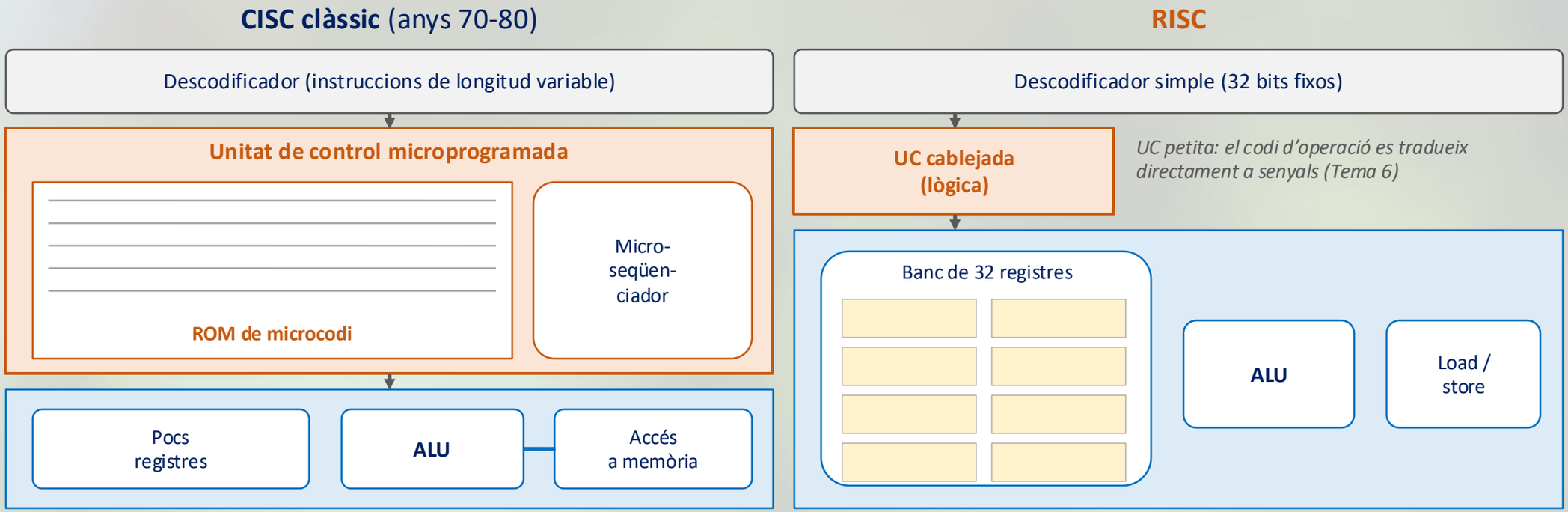
```
add dword ptr [ebx+8], 5
```

RISC-V (3 instruccions)

```
lw   x5, 8(x9)
addi x5, x5, 5
sw   x5, 8(x9)
```

El format fix de RISC-V està pensat per **simplificar el maquinari**: els camps (registres, opcode) sempre són al mateix lloc.

6. CISC i RISC (III): com són per dins



x86 actual
CISC per fora, RISC per dins

instrucció x86 → descodificador → micro-ops (µops) → nucli tipus RISC

Micro-op (µop): operació elemental de tipus RISC en què el descodificador trenca cada instrucció x86. Ex.: `add dword ptr [ebx+8], 5` → 3 µops: llegir memòria, sumar, escriure memòria (com les 3 instruccions RISC-V de la diapositiva anterior).

La descodificació és **cablejada** per a gairebé totes les instruccions; la ROM de microcodi només queda per a les més complexes.

6. CISC i RISC (IV): qui guanya?

T = N × CPI × Tc

CISC	N ↓ menys instruccions	CPI ↑ cada una fa més feina	Cap guanya per si sol: depèn del **producte** dels tres factors.
RISC	N ↑ més instruccions	CPI ≈ 1 i Tc més petit	

- Avui la frontera s'ha difuminat: els x86 tradueixen internament a micro-ops (RISC per dins).
- ARM domina mòbils i sistemes encastats; RISC-V creix en encastats, IoT i acceleradors.
- **Per què RISC-V a l'assignatura?**
  - ISA **oberta i lliure**: qualsevol la pot implementar sense pagar llicències.
  - **Modular**: base RV32I + extensions (M multiplicació, F/D coma flotant, C instruccions comprimides, V vectors…).
  - Simple i neta: ideal per aprendre. → **Tema 5**